



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-13

Experiment 13. Alpha-Beta Pruning

Aim

To implement the Alpha-Beta Pruning algorithm for gaming using Python.

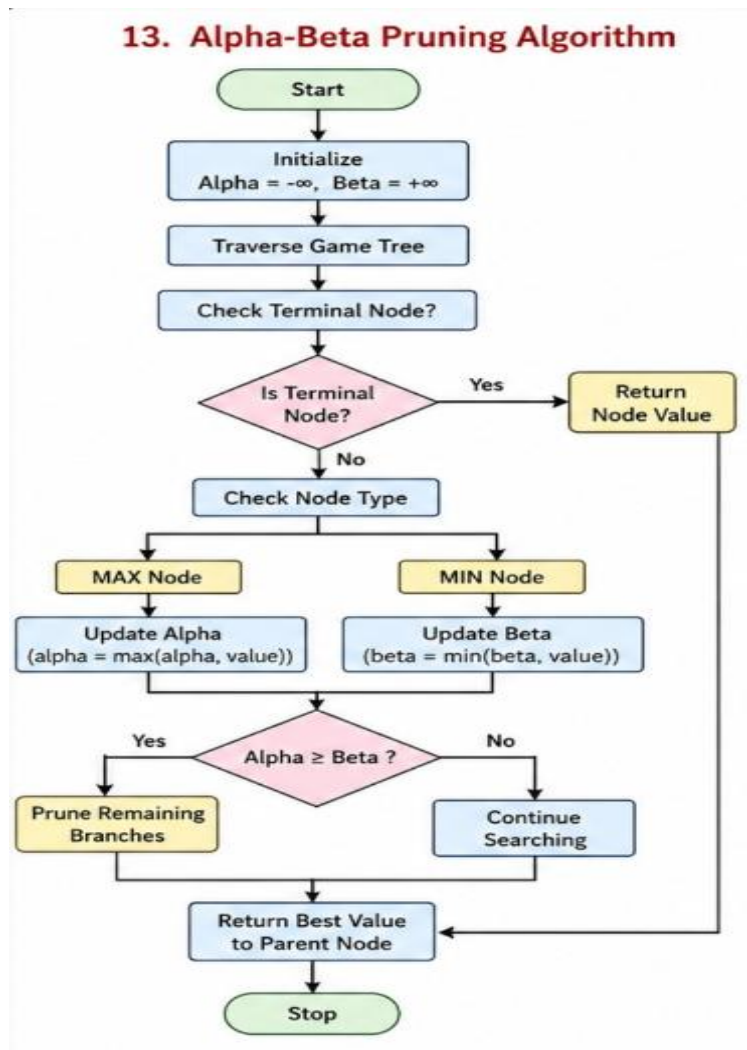
Objectives

- To understand Alpha-Beta pruning.
- To improve Minimax efficiency.
- To eliminate unnecessary branches.
- To find the optimal game decision.

Algorithm

1. Initialize Alpha to $-\infty$ and Beta to $+\infty$.
2. Traverse the game tree.
3. Update Alpha at MAX nodes.
4. Update Beta at MIN nodes.
5. Prune when Alpha $\geq$ Beta.
6. Return the best value.

Flowchart



Code

```
def alphabeta(depth, node,
max_player, values, alpha,
beta):

    if depth == 2:

        return values[node]
```

```
if max_player:

    best = float("-inf")

    for i in [0, 1]:

        best = max(best,
alphabetabeta(

            depth+1,
node*2+i, False,

            values, alpha,
beta))

        alpha = max(alpha,
best)

        if alpha >= beta:

            break

    return best

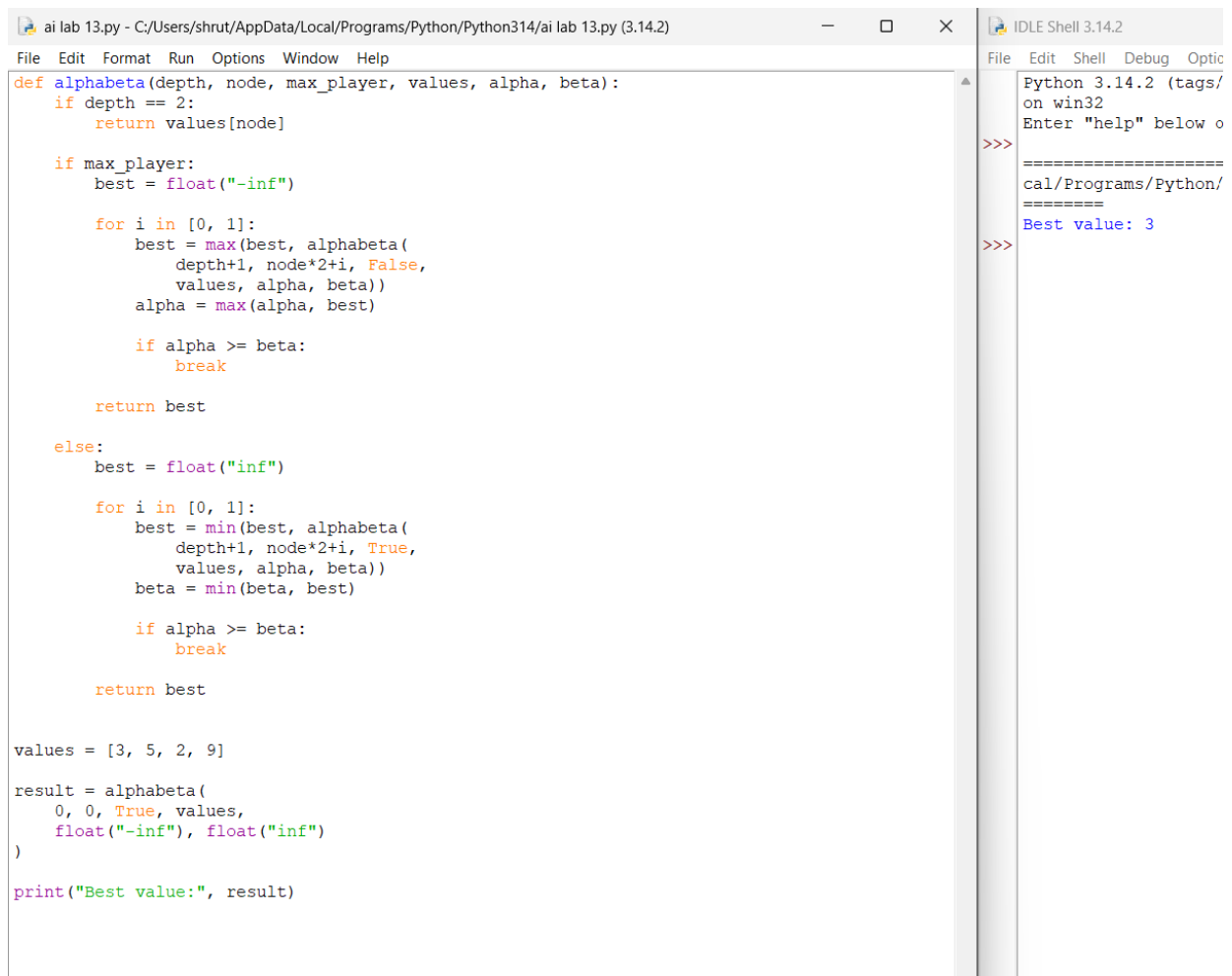
else:

    best = float("inf")

    for i in [0, 1]:
```

```
        best = min(best,
alphabeta(
        depth+1,
node*2+i, True,
        values, alpha,
beta))
        beta = min(beta,
best)
        if alpha >= beta:
            break
return best
values = [3, 5, 2, 9]
result = alphabeta(
    0, 0, True, values,
    float("-inf"), float("inf")
)print("Best value:", result)
```

Output



```
ai lab 13.py - C:/Users/shrut/AppData/Local/Programs/Python/Python314/ai lab 13.py (3.14.2)
File Edit Format Run Options Window Help
def alphabeta(depth, node, max_player, values, alpha, beta):
    if depth == 2:
        return values[node]

    if max_player:
        best = float("-inf")
        for i in [0, 1]:
            best = max(best, alphabeta(
                depth+1, node*2+i, False,
                values, alpha, beta))
            alpha = max(alpha, best)

            if alpha >= beta:
                break

        return best
    else:
        best = float("inf")
        for i in [0, 1]:
            best = min(best, alphabeta(
                depth+1, node*2+i, True,
                values, alpha, beta))
            beta = min(beta, best)

            if alpha >= beta:
                break

        return best

values = [3, 5, 2, 9]
result = alphabeta(
    0, 0, True, values,
    float("-inf"), float("inf")
)
print("Best value:", result)

Python 3.14.2 (tags/
on win32
Enter "help" below o
>>>
=====
cal/Programs/Python/
=====
Best value: 3
>>>
```

Result

The program successfully finds the optimal value using Alpha-Beta pruning.

Conclusion

Thus, the Alpha-Beta Pruning algorithm was successfully implemented, reducing unnecessary searches in the game tree.